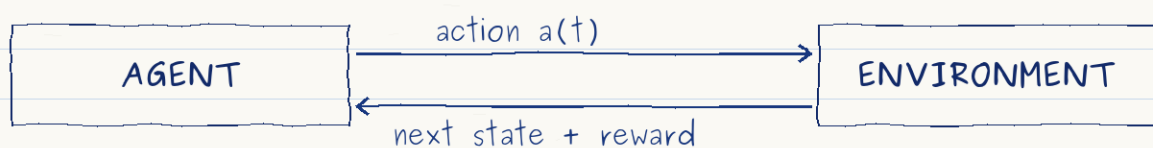


Q1 - Reinforcement Learning and data preprocessing

1) Reinforcement Learning (RL)

Reinforcement Learning is a learning framework in which an agent learns by interacting with an environment. At time t , the agent observes a state $s(t)$, chooses an action $a(t)$, receives a numerical reward $r(t+1)$, and moves to the next state $s(t+1)$. The objective is not merely to obtain the largest immediate reward; it is to learn a policy that maximizes the expected cumulative return over time.



- Agent: the decision maker. In the car example, it is the driving controller.
- Environment: everything with which the agent interacts, including the road and traffic.
- State: the information used for a decision, such as speed, lane position and nearby objects.
- Action: a control command such as steering, throttle, braking or a lane-change decision.
- Reward: scalar feedback that says whether the recent behaviour was desirable.
- Policy: a rule $\pi(a \text{ given } s)$ that maps a state to an action or an action probability.
- Return: $G(t) = r(t+1) + \gamma r(t+2) + \gamma^2 r(t+3) + \dots$, where γ discounts distant rewards.
- Value functions estimate long-term return. $Q(s, a)$ measures the value of taking action a in state s .

Q1 (cont.)

Learning requires a balance between exploration, which tries unfamiliar actions to collect information, and exploitation, which uses the best action currently known. Common approaches include Q-learning and Deep Q-Networks for action values, policy-gradient methods for directly improving a policy, and actor-critic methods that learn both a policy and a value estimate. Training may be organized into episodes, with terminal states such as reaching the destination or ending a simulation.

2) Autonomous-car environment

For an autonomous car, the environment contains the lane geometry, curves, intersections, traffic signs and lights, other vehicles, pedestrians, cyclists, obstacles, weather, road friction and the destination map. The observable state can be constructed from camera, LiDAR, radar, GPS and inertial sensors. Useful state variables include vehicle speed, heading, lane offset, distance and relative speed to nearby objects, signal status, free-space map and route progress.

The action space may be continuous, such as steering angle, throttle and brake pressure, or partly discrete, such as keep lane, change lane, stop and turn. Because real-world exploration is unsafe, most learning should first occur in a high-fidelity simulator, followed by carefully constrained testing and safety monitoring.

3) Reward design for the car

- Give a strong positive reward for safely reaching the destination.
- Give small positive rewards for forward route progress, correct lane position and maintaining a safe following distance.
- Reward obeying traffic lights, speed limits and right-of-way rules.
- Reward smooth steering, acceleration and braking so that passenger comfort and energy use are considered.
- Give a very large negative reward for a collision. Also penalize a near miss, lane departure, red-light violation, speeding, excessive delay and harsh control actions.
- Use terminal conditions for collision, an unrecoverable off-road state, or successful arrival.

Q1 (cont.)

Reward shaping *must* be designed carefully. If progress is rewarded without safety constraints, the car may learn to drive quickly but dangerously. A practical system therefore combines the learned objective with hard safety rules, constrained action limits and independent emergency braking.

4) Why preprocessing and feature transformation are essential

A machine-learning algorithm learns the patterns present in its input. If the input contains errors, incompatible scales or leakage, the model learns misleading patterns. Proper preprocessing makes the representation consistent with the assumptions of the algorithm and improves numerical stability, training speed and generalization.

- Clean missing values, impossible values, duplicates and inconsistent units. Missingness can be imputed and, when useful, represented by a missing-value indicator.
- Treat outliers using investigation, clipping, robust scaling or suitable transformations. A few extreme values can dominate a loss or a distance measure.
- Encode categorical variables correctly. One-hot, ordinal or target-based encodings should be chosen according to meaning and fitted without leaking the test labels.
- Scale numerical features. Standardization or normalization is crucial for KNN, K-Means, SVMs and gradient-based models because a large-unit feature can otherwise dominate.
- Transform skewed variables using log or power transformations. This can stabilize variance and make relationships easier for a model to learn.
- Construct meaningful features, remove irrelevant or duplicated variables, and reduce very high dimensionality when needed.
- Fit every learned transformation only on the training set, then apply exactly the same pipeline to validation, test and production data.

Correct order: split data → fit preprocessing on training data → transform all splits → train and evaluate.

Thus preprocessing is not cosmetic. It affects the geometry of the feature space, the conditioning of optimization, the validity of evaluation and finally the reliability of the trained model.

Q2 - OR alternative: customer segmentation and accuracy

1) Unsupervised customer segmentation

Because the company has no explicit labels such as a preferred product class or a satisfaction score, the task is to discover structure rather than predict a known target. A suitable solution is clustering, supported by customer-level feature engineering.

- Create one row per customer over a fixed time window.
- Use browsing features such as number of sessions, products viewed, category-view proportions, search count, dwell time and time since the last visit.
- Use purchase features such as recency, purchase frequency, monetary value, average basket size, categories purchased, repeat-purchase rate and return rate.
- Add behavioural ratios such as viewed-to-purchased conversion, cart-abandonment rate, discount usage and mobile-versus-desktop share.

Counts and monetary values are usually skewed, so log transformation can be applied. Missing values are handled, categorical information is encoded, and numerical features are standardized so that one high-magnitude variable does not control the distance. PCA or a learned embedding may be used to compress a very wide feature matrix.

K-Means is a simple starting method when roughly compact clusters are expected. Gaussian Mixture Models provide soft membership probabilities; hierarchical clustering gives a useful dendrogram; and DBSCAN can detect irregular dense groups and noise. The number of clusters can be selected using the elbow curve, silhouette score, stability across resamples and, most importantly, business interpretability.

Example segments and insights

- Loyal high-value customers: frequent purchases, high spend and short recency. They suit loyalty rewards, early access and retention attention.
- Active browsers with low conversion: many views and long sessions but few purchases. They reveal friction in price, trust, delivery or checkout.

Q2 (cont.)

- Bargain seekers: strong response to coupons and sale periods. They help plan promotion timing without giving unnecessary discounts to everyone.
- Category specialists: repeated interest in one category. They support focused recommendations, content and inventory decisions.
- New or infrequent customers: little history and long recency. Separate onboarding from re-engagement, and avoid overconfident personalization.

Unsupervised learning is appropriate because it can expose latent behavioural patterns without labelled outcomes. The company should profile every cluster, test whether it remains stable over time, and run controlled campaigns to verify that the segments are useful rather than merely mathematically separated.

2) Accuracy for the cat-dog model

There are 100 images. The model correctly classifies all 80 cats and correctly classifies none of the 20 dogs.

$$\text{Accuracy} = \text{number correct} / \text{total} = 80 / 100 = 0.80 = 80\%$$

Actual class	Predicted cat	Predicted dog
Cat (80)	80	0
Dog (20)	20	0

The result is not effective even though 80% appears high. The classes are imbalanced, and a trivial model that always says cat also obtains 80%. Dog recall is 0%, cat recall is 100%, and balanced accuracy is $(100\% + 0\%) / 2 = 50\%$. The confusion matrix and per-class precision, recall and F1 score are therefore necessary.

Q3 - Least-squares line and Decision Tree versus KNN

A) Least-squares estimate

Hours studied X	Test score Y
1	50
2	55
3	65

For the line $Y = a + bX$, use the least-squares formulas. Here $n = 3$, $\sum X = 6$, $\sum Y = 170$, $\sum XY = 355$ and $\sum X^2 = 14$.

$$b = [n \cdot \sum(XY) - \sum(X) \cdot \sum(Y)] / [n \cdot \sum(X^2) - (\sum X)^2]$$

$$b = [3 \cdot 355 - 6 \cdot 170] / [3 \cdot 14 - 6^2] = 45 / 6 = 7.5$$

$$a = [\sum(Y) - b \cdot \sum(X)] / n = [170 - 7.5 \cdot 6] / 3 = 41.6667$$

Therefore, the best-fit line is $Y = 41.67 + 7.50 X$.

Interpretation: within this small data set, one additional hour of study is associated with an estimated increase of 7.5 marks. The fitted scores for $X = 1, 2$ and 3 are approximately 49.17, 56.67 and 64.17.

B) When to choose a Decision Tree over KNN

- Choose a Decision Tree when an interpretable set of if-then rules is important. A path from root to leaf gives a direct explanation.
- Trees naturally learn nonlinear thresholds and feature interactions. They can use numerical and suitably encoded categorical features.
- A tree does not require feature scaling because splits depend on ordering rather than Euclidean distance. KNN is very sensitive to scale.
- Prediction with a trained tree is fast and requires only a sequence of comparisons. KNN must store the training set and calculate many distances for each new case.
- Trees are often preferable when there are irrelevant dimensions or when neighbourhood distance is not meaningful. KNN suffers in high-dimensional spaces.
- KNN may be preferred for a small, clean, low-dimensional data set with a smooth local decision boundary and enough memory for instance storage.

Q3 (cont.)

A single tree can overfit and be unstable, so depth limits, minimum leaf size or pruning should be used. If predictive accuracy is the main goal, a tree ensemble such as Random Forest may be stronger than one tree.

Q4 - OR alternative: ensembles and Logistic Regression

1) Why use an ensemble instead of one supervised model?

An ensemble combines predictions from several models. Its main advantage is that different models make different errors. If those errors are not perfectly correlated, combining the models can produce a more stable and accurate predictor than relying on one fitted hypothesis.

- Bagging trains models on resampled data and averages them. It mainly reduces variance. Random Forest also randomizes the features considered by each tree, increasing diversity.
- Boosting trains weak learners sequentially, giving later learners more attention to residual errors. It can reduce bias and build a strong nonlinear predictor.
- Voting combines class labels or probabilities from several models.
- Stacking learns a meta-model that decides how to combine heterogeneous base-model outputs.
- Averaging reduces sensitivity to a particular sample, initialization or noisy feature. It often gives smoother probabilities and better generalization.
- Different inductive biases can complement one another: for example, a linear model captures a broad trend while a tree captures interactions.

The benefit is greatest when the component models are individually competent and sufficiently diverse. Combining many almost identical, biased models adds little. Ensembles also increase training cost, memory use and deployment complexity, and they are usually less interpretable. Validation predictions must be produced without data leakage, especially in stacking.

2) Logistic Regression in real-life applications

Logistic Regression is a probabilistic classification model. For binary output, it models the probability of class 1 using the sigmoid function:

$$p(y=1 \text{ given } x) = 1 / [1 + \exp(-(b_0 + b_1x_1 + \dots + b_px_p))]$$

Equivalently, the log-odds are a linear function of the features.

Parameters are fitted by maximum likelihood, commonly by minimizing cross-entropy loss. L1 or L2 regularization controls complexity and can reduce overfitting.

- Credit risk: probability that a borrower will default.

Q4 (cont.)

- Health care: probability of disease, readmission or treatment response.
- Customer analytics: churn, subscription renewal or purchase conversion.
- Email and security: spam, fraud or suspicious-transaction classification.
- Operations: probability of equipment failure or late delivery.

Strengths

- Fast to train and predict, even for large sparse data sets.
- Outputs a probability, allowing a threshold to be chosen according to the cost of false positives and false negatives.
- Coefficients are interpretable: $\exp(b_j)$ is the multiplicative change in odds for a one-unit increase in feature j , holding other variables fixed.
- Regularization handles many features and provides a strong, transparent baseline.

Limitations and responsible use

- The decision boundary is linear in the supplied feature representation. Curves and interactions require transformed or interaction features.
- Strong multicollinearity makes individual coefficients unstable. Scaling, regularization and feature review are useful.
- Outliers, missing data and class imbalance must be handled. Accuracy alone may be misleading.
- A coefficient describes association under the fitted model; it does not by itself prove causation.
- Probabilities and thresholds should be checked on validation data using precision, recall, ROC-AUC, calibration and application-specific costs.

For more than two classes, one-versus-rest logistic models or multinomial softmax regression can be used. In practice, Logistic Regression is valuable when interpretability, probability estimates and efficient deployment matter.

Q5 - Preprocessing for clustering and one K-Means iteration

1) Impact of preprocessing on clustering

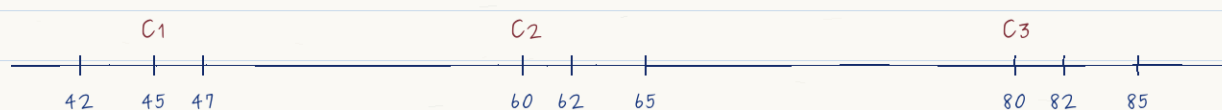
Clustering is driven directly by similarities or distances, so preprocessing can change the clusters more than the choice of algorithm. There is no target label to correct a poor representation.

- Scaling: with Euclidean distance, a feature measured in thousands dominates a feature measured between 0 and 1. Standardization, min-max scaling or robust scaling gives intended importance.
- Missing values: distance cannot be interpreted consistently when values are absent. Imputation and missingness indicators may be needed.
- Outliers: K-Means centroids are means and can be pulled far from the main group. Detect, cap, transform or use robust methods when justified.
- Categorical variables: arbitrary integer codes create false distances. Use suitable encodings or algorithms designed for mixed data.
- Skewness: log or power transformations reduce the dominance of long-tailed counts and spending values.
- Feature selection and dimensionality reduction: irrelevant dimensions hide neighbourhood structure. PCA can summarize correlated numerical features.
- Duplicates and sampling bias: repeated records or an unrepresentative sample can create artificial dense regions.

The entire preprocessing pipeline should be documented and fitted consistently. Cluster quality should then be checked by internal measures, stability and real-world interpretability.

2) One iteration of K-Means for the weights

Initial centroids are $C_1 = 45$, $C_2 = 60$ and $C_3 = 80$. Each one-dimensional weight is assigned to the closest centroid using absolute distance, which is Euclidean distance in one dimension.



Q5 (cont.)

Cluster	Assigned weights after first assignment	Updated centroid
C1	42, 45, 47	$(42+45+47)/3 = 44.67$
C2	60, 62, 65	$(60+62+65)/3 = 62.33$
C3	80, 82, 85	$(80+82+85)/3 = 82.33$

After one iteration: $C1 = 44.67$, $C2 = 62.33$, $C3 = 82.33$.

Q6 - OR alternative: two-cluster K-Means and system design

1) K-Means for {2, 4, 10, 12, 3, 20, 30, 11, 25}

For one-dimensional data, assign each value to the nearest centroid and then replace each centroid by the mean of its assigned values.

Iteration 1: start $C_1 = 4$, $C_2 = 11$

- Cluster 1 = {2, 4, 3}; new $C_1 = (2+4+3)/3 = 3$.
- Cluster 2 = {10, 12, 20, 30, 11, 25}; new $C_2 = 108/6 = 18$.

Iteration 2: centroids 3 and 18

- Cluster 1 = {2, 4, 10, 3}; new $C_1 = 19/4 = 4.75$.
- Cluster 2 = {12, 20, 30, 11, 25}; new $C_2 = 98/5 = 19.6$.

Iteration 3: centroids 4.75 and 19.6

- Cluster 1 = {2, 4, 10, 12, 3, 11}; new $C_1 = 42/6 = 7$.
- Cluster 2 = {20, 30, 25}; new $C_2 = 75/3 = 25$.

Iteration 4: centroids 7 and 25

The assignments remain unchanged, so the algorithm has converged.

Final $C_1 = \{2, 3, 4, 10, 11, 12\}$, centroid = 7.

Final $C_2 = \{20, 25, 30\}$, centroid = 25.

2) Design of a clustering-based hospital care system

Goal: segment patients into clinically meaningful utilization and risk profiles so that a hospital can plan preventive care. The clusters are decision-support groups, not diagnoses.

- Data: age band, BMI, blood pressure, glucose, chronic-condition counts, medication count, emergency visits, admissions, missed appointments, laboratory summaries and activity measures. Personally identifying fields are excluded from modelling.
- Preprocessing: validate units, impute missing values, cap implausible outliers, one-hot encode categories, transform skewed visit counts and standardize numerical features. Feature windows must refer only to information available before the intervention date.
- Modelling: begin with K-Means for transparent centroid profiles. Compare Gaussian mixtures for overlapping groups and hierarchical clustering for nested structure. Run several initializations.

Q6 (cont.)

- Model selection: compare silhouette score, within-cluster dispersion, stability under resampling and review by clinicians. A mathematically compact cluster is useful only when its profile is understandable and actionable.
- Possible outputs: stable low-use patients; chronic-condition patients needing regular monitoring; high emergency-use patients; missed-appointment or adherence-risk patients; and recently deteriorating patients.
- Action layer: assign tailored reminders, care-manager calls, telehealth monitoring, nutrition support or medication review. Measure outcomes through controlled evaluation rather than assuming the clusters cause improvement.
- Governance: protect privacy, audit subgroup performance, avoid stigmatizing labels, allow clinical override and retrain when population or care pathways change.

The deployed system should show a patient profile and the nearest cluster centroid, together with uncertainty and the date of the last model update. This keeps the clustering result interpretable and supports responsible human decisions.